Deploy Project using ECR

Stage 01

- Go to AWS Console and open: ECR
- Make sure ur region its must be important when we create EC2 instance
- Now create ECR for ur project
- Create 2 ECR repo for **backend** and **frontend**
- After creating, you will see something like:
    1. 123456789.dkr.ecr.ap-south-1.amazonaws.com/backend
    2. 123456789.dkr.ecr.ap-south-1.amazonaws.com/frontend
- U have to save this in notepad (optional) but if not u can see it after going ECR on AWS

Stage 02

- You only need to do 2 things here
    1. Create IAM User → for Jenkins (push to ECR)
    2. Create IAM Role → for EC2 (pull from ECR)
- <u>Create IAM user for Jenkins</u>
    1. Go to AWS → IAM
    2. Click "Users" → "Create user" - **jenkins-ecr-user**
    3. Set permissions - Select: **Attach policies directly**
    4. Select **AmazonEC2ContainerRegistryFullAccess**
    5. Next → Create user
- Now Create Access key
    1. After user created Go inside the **user** and Click **Security credentials** and **Scroll → Access keys → Create key**
    2. Choose: **Command Line Interface (CLI)**
- Copy these IDs – IMPORTANT (COPY THIS CREDENTIALS INTO NOTEPAD)
    1. Access Key ID
    2. Secret Access Key
- <u>Create IAM Roles for Jenkins</u>
    1. Go to "Roles" → Create role
    2. Select: AWS service and EC2
- Attach permission
    1. Select **AmazonEC2ContainerRegistryFullAccess**
    2. Name the role **EC2-ECR-Role**

Stage 03

- Now create EC2 instance
- Now let's attach to ROLE into EC2
    1. Go to: EC2 → your instance
    2. Actions → Security → Modify IAM role
    3. Select: **EC2-ECR-Role**

Stage 04

- Go to Jenkins and install this plugin **AWS Credentials**
- After install go to credentials in Jenkins select **AWS credentials**
    1. Username = **AWS Access Key ID** – {IAM user credentials}
    2. Password = **AWS Secret Access Key**
    3. ID = aws-creds (u can change anything to it)
- Go to terminal and run **docker –version** u must see version **(if not install docker)**
- Also run **aws --version**
- If not install go to this link and download after install it https://awscli.amazonaws.com/AWSCLIV2.msi
- And then go to CLI file path **(usually C:\Program Files\Amazon\AWSCLIV2)**
- Now go to **environment variable** in the and set **environment variable** after choose **path** click **new** and **add that file path** after click all windows OK button)
- Now check terminal (CMD) **aws –version**
- Configure Aws in Jenkins User Context
    1. Open CMD and run: **aws configure**
    2. AWS Access Key ID → your IAM key
       AWS Secret Access Key → your IAM secret
       Region → us-east-1
       Output format → json
    3. Now test: **aws sts get-caller-identity**

Stage 05

- Create Jenkins file

```
stage('Docker Build Images') {
    steps {
        bat 'docker build -t backend ./backend'
        bat 'docker build -t frontend ./frontend'
    }
}

stage('Trivy Scan Backend') {
    steps {
        bat "\"${TRIVY_PATH}\" image --severity HIGH,CRITICAL backend"
    }
}

stage('Trivy Scan Frontend') {
    steps {
        bat "\"${TRIVY_PATH}\" image --severity HIGH,CRITICAL frontend"
    }
}

stage('ECR Login') {
    steps {
        withCredentials([aws(accessKeyVariable: 'AWS_ACCESS_KEY_ID', credentialsId: 'aws-creds', secretKeyVariable: 'AWS_SECRET_ACCESS_KEY')]){
        bat """
        aws ecr get-login-password --region %AWS_REGION% | docker login --username AWS --password-stdin %ECR_URL%
        """
        }
    }
}
```

- Add like this environment top of code (adjust ur region and ECR URL)

```
pipeline {
    agent any

    environment {
        AWS_REGION = "us-east-1"
        ECR_URL = "313772261399.dkr.ecr.us-east-1.amazonaws.com"

        BACKEND_IMAGE = "backend"
        FRONTEND_IMAGE = "frontend"

        GITLEAKS_PATH = "C:\\DevSecOps\\gitleaks\\gitleaks.exe"
        SONAR_SCANNER = "C:\\DevSecOps\\sonar-scanner\\bin\\sonar-scanner.bat"
        TRIVY_PATH = "C:\\DevSecOps\\trivy\\trivy.exe"
    }
```

- then build docker image (images name same as top of code) and scan each images also then login ECR (IMPORTANT)
- Set credential using pipelines syntax (use aws credential) after u can connect ur ECR

- To connect ECR use this **aws ecr get-login-password --region <Your Region> | docker login --username AWS --password-stdin <ECR_URL>**

```
        stage('Push Backend to ECR') {
            steps {
                bat """
                docker tag backend:latest %ECR_URL%/%BACKEND_IMAGE%:latest
                docker push %ECR_URL%/%BACKEND_IMAGE%:latest
                """
            }
        }

        stage('Push Frontend to ECR') {
            steps {
                bat """
                docker tag frontend:latest %ECR_URL%/%FRONTEND_IMAGE%:latest
                docker push %ECR_URL%/%FRONTEND_IMAGE%:latest
                """
            }
        }
    }
}
```

- Now we can push our images into ECR
- **Note:** Before push images make sure ur images optimized (if not ur images become huge size (1GB+)

Stage 06

- Install **Docker** and **awscli** in EC2
- Go to EC2 server using **ssh**
    1. sudo apt update
    2. sudo apt install docker.io -y
- Then start Docker:
    1. sudo systemctl start docker
    2. sudo systemctl enable docker
    3. docker –version
- Next Check — Aws Cli
    1. Type **aws –version** (If version not found)
    2. curl "https://awscli.amazonaws.com/awscli-exe-linux-x86_64.zip" -o "awscliv2.zip"
    3. sudo apt update
    4. check using **ls** if file is exit
    5. install unzip package **sudo apt install unzip -y**
    6. type **unzip awscliv2.zip**
    7. Now **sudo ./aws/install**
    8. Now check **aws –version**
- Install docker compose
    1. sudo apt update
    2. sudo apt install ca-certificates curl gnupg lsb-release -y
    3. sudo install -m 0755 -d /etc/apt/keyrings
    4. curl -fsSL https://download.docker.com/linux/ubuntu/gpg | \
       sudo gpg --dearmor -o /etc/apt/keyrings/docker.gpg
    5. echo \
       "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.gpg] \
       https://download.docker.com/linux/ubuntu $(lsb_release -cs) stable" | \
       sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
    6. sudo apt update
    7. sudo apt install docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin -y
    8. docker compose version
- Then check docker compose version **docker compose version**

Stage 07

- Now Install git on EC2 **sudo apt install git -y**
- Clone the repo **git clone <repo URL>**
- Go to ur folder **cd <ur folder name>**
- Edit docker compose file **nano docker-compose.yml (or u can edited on locllay after push images into ECR)**
- Now Login To ECR (Very Important)
    1. **aws ecr get-login-password --region <ur region>| docker login --username AWS --password-stdin <ECR URL>**
- Must show: **Login Succeeded**

Optional

- Sometime docker services are stop so then u have to start docker service **sudo systemctl start docker**
- Now Enable Docker on boot **sudo systemctl enable docker**
    2. Or u can restart docker **sudo systemctl restart docker**
- Check status **sudo systemctl status docker**
- Check using **docker ps**
- Then u can login ECR again

- Now u can pull images **docker compose pull**
- Now up container docker **compose up -d**
- U can check ECR repos using **aws ecr describe-repositories --region us-east-1**